

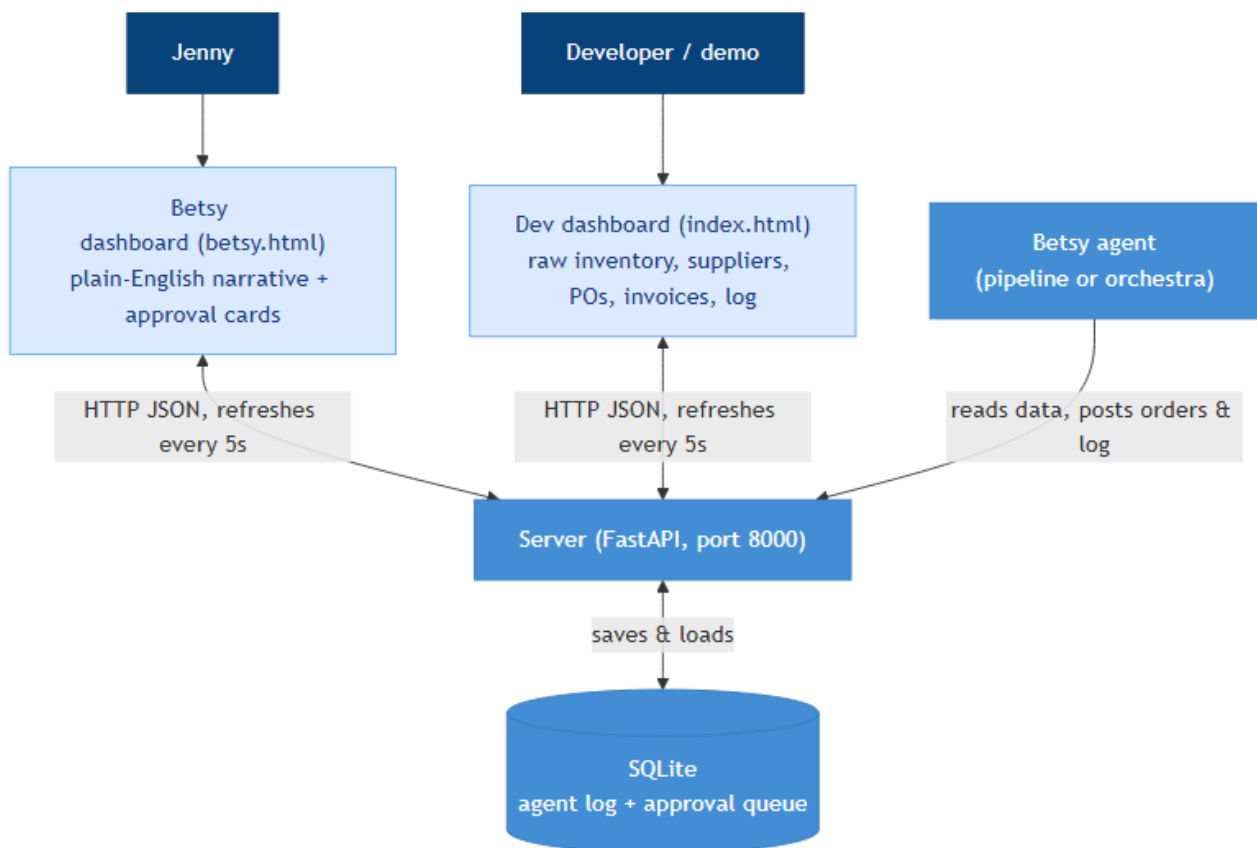
BETSY — THE CONTROL LAYER

The two screens, and how Betsy's actions show up in them

WHAT THIS IS

Betsy is built as a layer on top of a normal procurement system, not as a replacement for it. There are two screens, and both talk to the same server. This document explains what each screen is for and how an action Betsy takes turns into something a person can see.

Two screens, one server behind them



See the picture: *diagrams/api-two-surfaces.mmd*

THE TWO SCREENS

- The Betsy dashboard [dashboard/betsy.html] is Jenny's screen. It shows a plain-English summary of what Betsy did, the risk on each item, and the approval cards for anything that needs her decision. No raw IDs, no jargon.
- The Dev dashboard [dashboard/index.html] is the raw view for developers and demos. It shows the inventory table, the suppliers, the purchase orders, the invoices, and the full agent log.

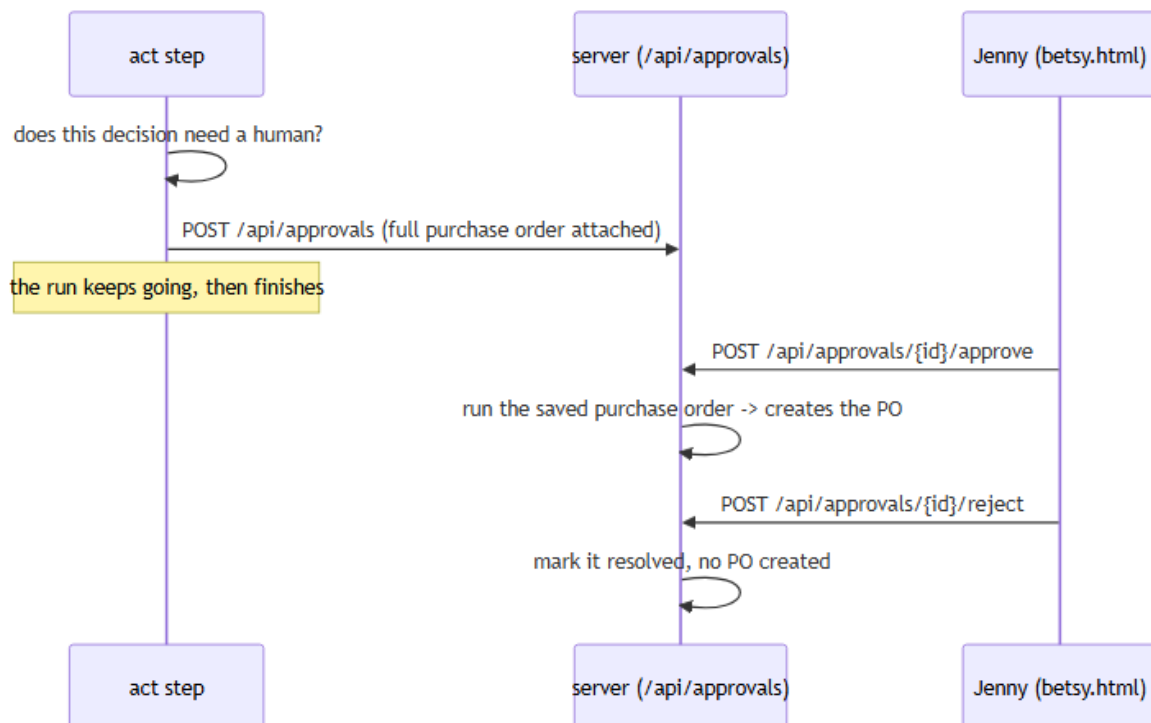
Both screens read from and write to the same server [server/main.py], and both refresh every five seconds. So an action taken on one screen shows up on the other on its next refresh. If Betsy were switched off, the screens and the data behind them would still work normally — that is the whole point of building Betsy as a layer.

HOW AN ACTION BECOMES VISIBLE

Each kind of decision Betsy makes maps to a specific change a person can see:

- A routine order Betsy can place itself becomes a new purchase order (POST to the purchase-orders endpoint), which appears in the Dev dashboard's PO list and in Jenny's narrative as "I ordered X from Y."
- An order that needs a person is placed on the approvals queue (POST to the approvals endpoint) and shows up as a card in Jenny's "needs your OK" section. When she approves it, the server places the saved order and the PO appears in the Dev dashboard.
- A flagged duplicate invoice is written to the agent log and highlighted in the invoice list; Jenny's narrative notes it.
- Every run, whatever happens, writes an entry to the agent log.

The human approve/reject round-trip is the same one used by both agents.



See the picture: *diagrams/pipeline-approval.mmd*

THE ENDPOINTS, IN PLAIN TERMS

The screens and the agents read through these:

- inventory, suppliers, purchase orders, invoices, the agent log, the pending approvals, and the active scenario.

The agents and the approve/reject buttons write through these:

- create a purchase order, write an agent-log entry, add an approval, approve or reject an approval, and (for demos) inject or reset a scenario.

The full endpoint list and exact paths are in the server routers [server/routers/].

WHY THE TWO SCREENS ARE SEPARATE

The two screens share the same data but serve different people. Developers and demos need the raw numbers and the ability to inject test scenarios; Jenny needs plain language and clear actions. Keeping them separate means each audience gets the view that suits it, while there is still only one set of data underneath. Betsy's only footprint on that data is the agent- log entries it writes and the purchase orders it places.

WHERE THIS LIVES IN THE CODE

- dashboard/betsy.html Jenny's plain-English screen
- dashboard/index.html the raw developer/demo screen
- server/main.py the server both screens talk to
- server/routers/ the individual endpoints (inventory, approvals, ...)
- server/db.py the SQLite store behind the log and approvals

DIAGRAMS FOR THIS DOCUMENT

- diagrams/api-two-surfaces.mmd the two screens and the server behind them
- diagrams/pipeline-approval.mmd the approve/reject round-trip (shared with the pipeline)